VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgaum-590018

Mini Project Report

on

"AI Blind Assistance Glasses"

Submitted in partial fulfillment of the requirements for the
Second Semester of the Bachelor of Engineering Degree, towards the completion of the **Mini Project** under the **Interdisciplinary Project-Based Learning**,
Department of Basic Sciences.

by

SN	Name	USN/Roll number
1	Sudiksha Lande	1CR25CD030
2	Syed Sameena	1CR25EC218
3	Syed Fateh Ali	1CR25EC217
4	Kushal Ranganathappa	1CR25CD029

Under the Guidance of
Professor Kashif Ahmed, Dept. of ECE

CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI,

BANGALORE-560037

CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI,

BANGALORE-560037

DEPARTMENT OF BASIC SCIENCES

CERTIFICATE

This is to certify that the File Structures mini project entitled "AI Blind Assistance Glasses" has been successfully carried out by Sudiksha Lande (1CR25CD030), Syed Sameena (1CR25EC218), Syed Fateh Ali (1CR25EC217) and Kushal Ranganathappa (1CR25CD029), bonafide students of **CMR Institute of Technology.**

The project is submitted in partial fulfillment of the requirements for the Second Semester of the Bachelor of Engineering Degree, towards the completion of the Mini Project under the **Interdisciplinary Project-Based Learning, Department of Basic Sciences.**

It is further certified that all corrections and suggestions indicated during the Internal Assessment have been duly incorporated in the project report submitted to the departmental library. This File Structures mini project report has been reviewed and approved as it satisfies the academic requirements prescribed for the said degree.

----------	**----------**
Signature of Guide	**Signature of HOD**
Professor **Kashif Ahmed** **Dept. of ECE, CMRIT**	**Dr. Raveesha K H** **HoD,** **Dept. of Physics, CMRIT**

External Viva

Name of the examiners — Signature with date

1.

2.

ACKNOWLEDGEMENT

I sincerely express my gratitude to **Dr. Sanjay Jain**, Principal, CMR Institute of Technology, Bangalore, for providing a supportive academic environment.

I extend my thanks to **Dr. Raveesha K H**, **HoD, Dept. of Physics, CMRIT** for his valuable guidance and support.

I am especially grateful to my internal guide, **Professor Kashif Ahmed, Department of Electronics and Communication Engineering**, for her/his constant encouragement and guidance throughout this project.

I also thank all the faculty members, non-teaching staff, and others who contributed directly or indirectly to the successful completion of this work.

Sudiksha Lande(1CR25CD030),

Syed Sameena (1CR25EC218),

Syed Fateh Ali (1CR25EC217),

Kushal Ranganathappa (1CR25CD029)

Abstract

Smart Blind Assistance Glasses is a low-cost wearable assistive technology prototype designed to support visually impaired users by detecting nearby obstacles and providing immediate audio or buzzerbased feedback. The system uses an ESP32 DevKit as the main controller, an HC-SR04 ultrasonic sensor for distance measurement, a buzzer for dynamic proximity alerts, and a DFPlayer Mini module for voice feedback. The ultrasonic sensor estimates distance by measuring the travel time of reflected sound waves, and the ESP32 maps this distance to a human-friendly alert pattern where closer obstacles produce faster beeping. The proposed extended system also includes an ESP32-CAM module for camera-based capabilities such as face recognition, text reading, and common object identification through external phone or laptop-based AI processing. The prototype demonstrates the feasibility of building an affordable smart wearable system within a student-project budget while emphasizing modular design, accessibility, real-time sensing, and future expandability.

Keywords: ESP32, assistive technology, ultrasonic sensor, smart glasses, DFPlayer Mini, ESP32-CAM, obstacle detection

TABLE OF CONTENT

Title **Page No**

Acknowledgement ..... **i**

Abstract ..... **ii**

List of Figures ..... **v**

Chapter 1

INTRODUCTION 1

1.1 Background..... 1

1.2 Need for Assistive Wearable Technology..... 1

1.3 Scope of the Project..... 1

Chapter 2

ABOUT THE PROJECT 2-3

2.1 Problem Statement 2

2.2 Objective of the Project 2

2.3 Proposed Solution 2

2.4 Advantages of Proposed solution 3

Chapter 3

DESIGN 4-10

3.1 Design Thinking Process 4

3.2 Methodology..... 4

3.3 Materials Used..... 5

3.4 System Architecture..... 6-7

3.5 Prototype..... 7-9

3.6 Circuit Connection Summary..... 10

Chapter 4

CODE SNIPPETS AND DESCRIPTION ……. 11-20

4.1 Program Codes……..…….. 11-18

4.2 Explanaions of Program Codes……... 19-20

Chapter 5

RESULT AND ANALYSIS ……. 21-23

5.1 Result ……..... 21

5.2 Analysis ……..... 22

5.3 User Testing and Feedback Plan……. 22-23

Chapter 6

CONCLUSION ……24-25

REFERENCES ……25-26

LIST OF FIGURES

Figure No.	Title	Page No.
Fig 3.2	Working Flow of Prototype	4
Fig 3.3	System Design and Project Workflow	6
Fig 3.4	System Architecture	7
Fig 3.5	Hardware Prototype Output	7-9

Chapter 1

INTRODUCTION

1.1 Background

Visual impairment creates daily mobility challenges, especially in unfamiliar indoor and outdoor environments. Conventional aids such as the white cane are useful for detecting ground-level obstacles, but they cannot easily detect obstacles at head or chest level. With the growth of low-cost embedded systems, sensors, and wireless microcontrollers, it is possible to create affordable assistive devices that provide real-time awareness to users.

The Smart Blind Assistance Glasses project explores a wearable prototype that uses an ESP32-based embedded system to sense obstacles and notify the user through buzzer and voice feedback. The design focuses on affordability, modularity, and ease of implementation for an engineering mini project.

1.2 Need for Assistive Wearable Technology

- To improve obstacle awareness for visually impaired users.
- To provide immediate feedback without requiring visual attention.
- To support low-cost accessibility using commonly available components.
- To create a scalable system that can later include face recognition, OCR, and object detection.

1.3 Scope of the Project

The current hardware prototype implements obstacle detection using an ultrasonic sensor, dynamic buzzer alerts, and DFPlayer Mini-based voice feedback logic. The ESP32-CAM module is planned as a vision extension for face recognition, text reading, and object identification.Since advanced image recognition requires higher processing power than the ESP32-CAM alone can reliably provide, the proposed design uses camera streaming with phone or laptop-based AI processing.

Chapter 2

Problem Statement

2.1 Problem Statement

Visually impaired individuals may face difficulty detecting nearby obstacles, identifying familiar people, reading short text, and recognizing common objects. Existing smart assistive devices are often costly or complex. Therefore, there is a need for a low-cost wearable prototype that can detect obstacles in real time and provide simple audio feedback, while also supporting future camera-based AI features.

2.2 Objectives of the Project

- Design a wearable glasses-based prototype for obstacle awareness.
- Measure obstacle distance using an ultrasonic sensor connected to ESP32.
- Generate intuitive buzzer patterns based on distance.
- Add voice feedback using DFPlayer Mini.
- Prepare ESP32-CAM integration for face recognition, OCR, and object recognition.
- Keep the target cost within approximately Rs. 2,000 – Rs. 3,500.

2.3 Proposed Solution

The proposed system consists of an ESP32 DevKit as the control unit, an HC-SR04 ultrasonic sensor for obstacle distance detection, a buzzer for real-time proximity feedback, and a DFPlayer Mini for recorded voice messages.

The camera-based functions are handled by ESP32-CAM and external AI software running on a phone or laptop. This modular approach avoids overloading a single microcontroller and improves reliability.

2.4 Advantages of the Proposed Solution

- Low-cost and suitable for student-level prototyping.
- Portable and wearable design.
- Real-time response to obstacles.
- Expandable architecture for camera and AI features.
- Uses common components available in India.
- Can be tested module-by-module for easier debugging.

Chapter 3

3.1 Design Thinking Process

The project follows the design thinking approach: empathize, define, ideate, prototype, and test. The need was identified from the mobility challenges faced by visually impaired users.

The solution was defined as a low-cost wearable device that gives non-visual feedback. Multiple design ideas were considered, including camera-only recognition and sensor-only detection. The final prototype uses a modular hybrid approach: local obstacle detection on ESP32 and camera-based AI as an extension.

3.2 Methodology

The implementation was divided into small testable stages. First, the ESP32 DevKit was configured in Arduino IDE and tested using Serial Monitor. Next, the HC-SR04 ultrasonic sensor was connected and tested for distance readings.

Then a buzzer was added and programmed to produce dynamic beep patterns. Finally, the DFPlayer Mini module was configured for voice feedback. ESP32-CAM is intended to stream images for AI-based recognition features.

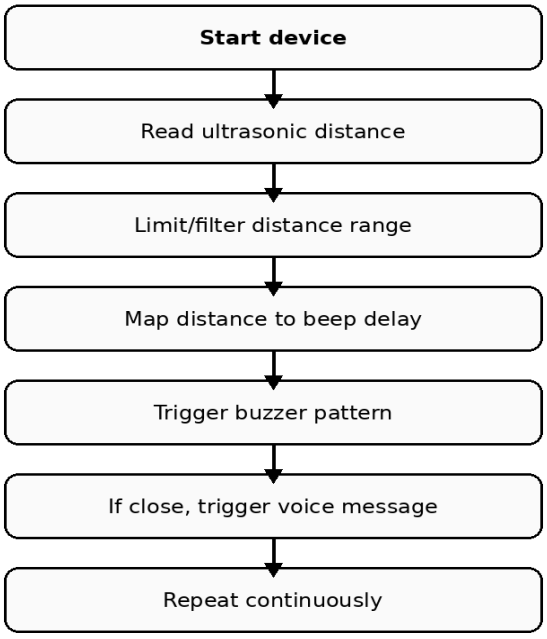
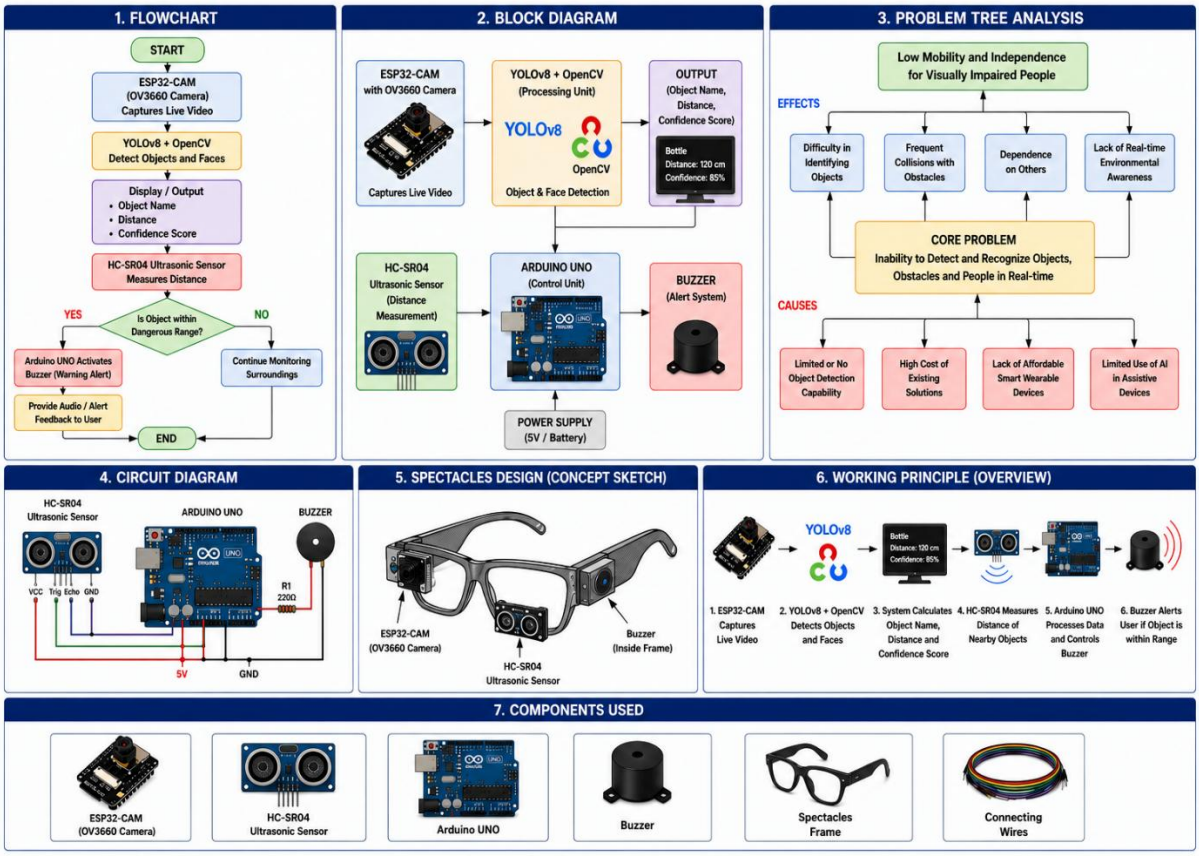


Fig. 3.2 Working Flow of the Prototype

3.3 Materials Used

Component	Purpose	Key Specification / Note
ESP32 DevKit	Main microcontroller used to read sensor data and control buzzer/DFPlayer	Wi-Fi/Bluetooth capable, USB programmable
HC-SR04 Ultrasonic Sensor	Measures distance to nearby obstacles	Range approximately 2 cm to 400 cm
Buzzer	Provides immediate sound alert	Two-pin active buzzer
DFPlayer Mini	Plays pre-recorded MP3 voice messages	Uses microSD card and UART
MicroSD Card	Stores voice feedback files	Files named 0001.mp3 etc.
ESP32-CAM	Camera module for future vision features	OV2640 camera, Wi-Fi streaming
18650 Battery + TP4056	Portable power supply	Rechargeable-power module
Spectacle Frame	Wearable base structure	Sensor mounted on frame

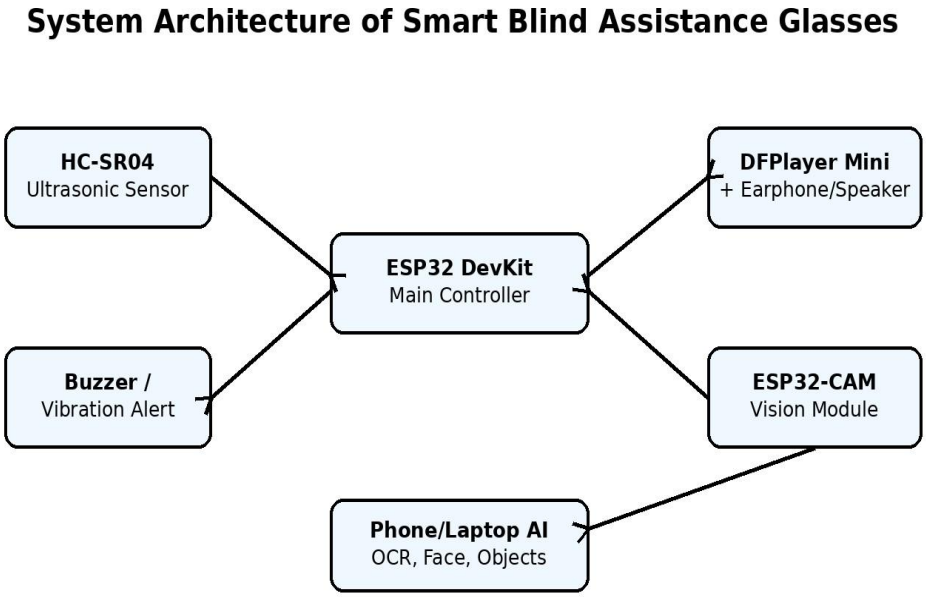
System Design and Project Workflow



3.4 System Architecture

The system architecture includes the ESP32 DevKit connected to the HC-SR04 ultrasonic sensor, buzzer, DFPlayer Mini, and ESP32-CAM module. The ultrasonic sensor detects nearby obstacles and sends data to the ESP32. Based on the measured distance, the ESP32 activates the buzzer and DFPlayer Mini for audio feedback.

The ESP32-CAM streams images for advanced AI processing such as OCR, face recognition, and object detection through external devices like laptops or smartphones.



3.5 Prototype

The following figures show the developed AI Blind Assistance Glasses prototype with integrated camera module, ultrasonic sensor, Arduino Uno, and circuit connections.

Image 1 – Front View of Smart Blind Glasses

Image 2 – Angular View Showing Arduino Uno and Circuit Connections

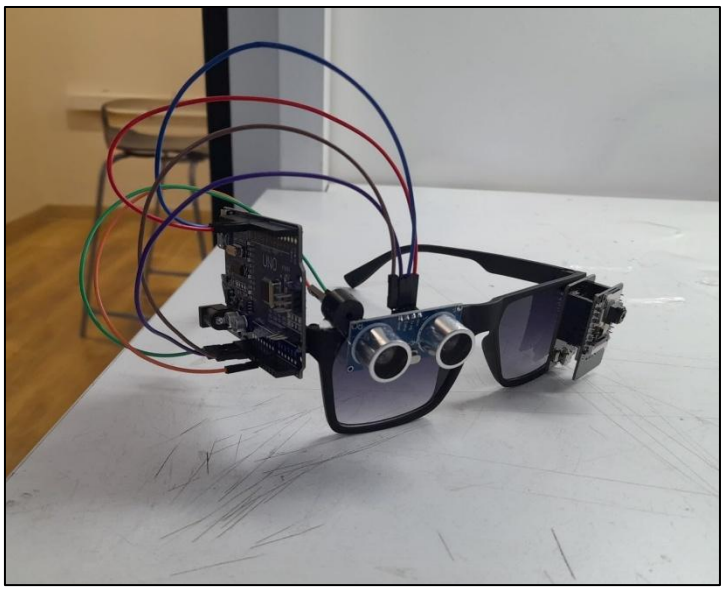
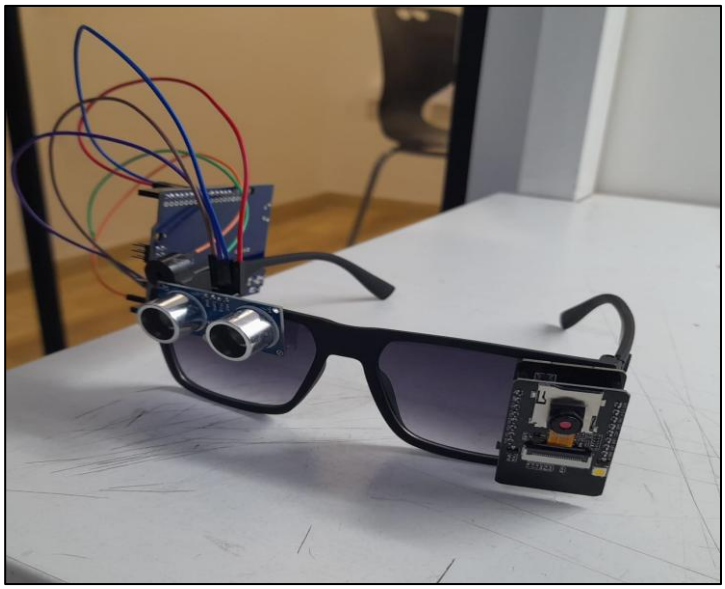
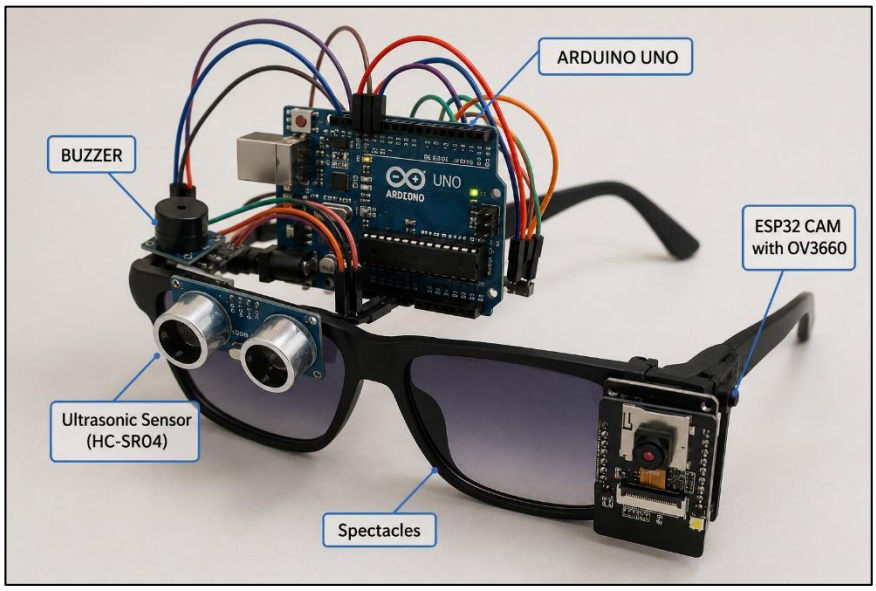


Image 3 – Side View Showing ESP32-CAM Module Integration



Prototype Description



The prototype model consists of smart glasses integrated with an ESP32-CAM module for image capture and an ultrasonic sensor for obstacle detection. The Arduino Uno board controls the connected components and processes sensor data. The system performs real-time object detection using YOLOv8 and provides audio feedback and buzzer alerts to assist visually impaired users during navigation.

3.6 Circuit Connection Summary

Module Pin	ESP32 Pin	Description
HC-SR04 VCC	VIN / 5V	Power supply for sensor
HC-SR04 GND	GND	Common ground
HC-SR04 TRIG	GPIO 5	Trigger pulse from ESP32
HC-SR04 ECHO	GPIO 18	Echo signal to ESP32
Buzzer +	GPIO 23	Buzzer control output
Buzzer –	GND	Common ground
DFPlayer VCC	VIN / 5V	Power supply
DFPlayer GND	GND	Common ground
DFPlayer TX	GPIO16	Serial data from DFPlayer
DFPlayer RX	GPIO17	Serial command from ESP32
Speaker/Earphone	SPK1 and SPK2	Audio output from DFPlayer

Chapter 4

Implementation

Program Codes:

Object Detection

```
from ultralytics import YOLO
import requests
import PIL.Image
import io
import time
import subprocess
import datetime
import numpy as np
import keyboard
import cv2
import threading  # <-- NEW

ESP32_IP = "http://10.202.0.249"
model = YOLO("yolov8n.pt")
last_detected = ""

KNOWN_HEIGHTS_CM = {
    "person": 170, "chair": 90, "bottle": 25, "cup": 12,
    "laptop": 30, "cell phone": 15, "book": 25, "backpack": 50,
    "bicycle": 100, "car": 150, "dog": 60, "cat": 25,
    "tv": 60, "mouse": 4, "keyboard": 3, "clock": 30,
}
FOCAL_LENGTH = 600
FRAME_HEIGHT = 480

# ============================
# SHARED DISPLAY FRAME
# Updated by detection loop, read by display thread
# ============================
current_display_frame = None
frame_lock = threading.Lock()
stop_display = False
```

```
# ============================
# DISPLAY THREAD
# Runs at ~30fps independently of detection speed
# ============================
def display_thread():
    global stop_display
    window_name = "Blind Assistant - Live View"
    cv2.namedWindow(window_name, cv2.WINDOW_NORMAL)
    cv2.resizeWindow(window_name, 800, 600)

    while not stop_display:
        with frame_lock:
            frame = current_display_frame.copy() if current_display_frame is not None else None

        if frame is not None:
            cv2.imshow(window_name, frame)
        else:
            # Show placeholder until first frame arrives
            placeholder = np.zeros((480, 640, 3), dtype=np.uint8)
            cv2.putText(placeholder, "Waiting for camera...", (160, 240),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.9, (100, 100, 100), 2)
            cv2.imshow(window_name, placeholder)

        key = cv2.waitKey(33)  # ~30fps
        if key == 27:  # ESC to close window
            stop_display = True
            break

    cv2.destroyAllWindows()

# ============================
# UPDATE DISPLAY FRAME (called from main loop)
# ============================
def update_display(pil_image, detected):
    frame = cv2.cvtColor(np.array(pil_image), cv2.COLOR_RGB2BGR)

    if detected:
        for label, info in detected.items():
            x1, y1, x2, y2 = info['box']
            conf = info['confidence']
            dist = info['distance_cm']
```

```
        source = info.get('dist_source', 'vision')

        color = (0, 200, 0) if source == "sensor" else (0, 140, 255)
        cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)

        if dist:
            dist_m = dist / 100
            dist_str = f"{dist}cm" if dist_m < 1.0 else f"{dist_m:.1f}m"
            tag = f"{label} {conf}% | {dist_str}"
        else:
            tag = f"{label} {conf}%"

        (tw, th), _ = cv2.getTextSize(tag, cv2.FONT_HERSHEY_SIMPLEX,
0.55, 1)
        cv2.rectangle(frame, (x1, y1 - th - 8), (x1 + tw + 6, y1), color, -1)
        cv2.putText(frame, tag, (x1 + 3, y1 - 5),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255, 255), 1,
cv2.LINE_AA)

    ts = datetime.datetime.now().strftime("%H:%M:%S")
    count = len(detected) if detected else 0
    status = f"[{ts}] Objects: {count} | ESC to close window | Ctrl+C to quit"
    cv2.rectangle(frame, (0, frame.shape[0] - 26), (frame.shape[1],
frame.shape[0]), (20, 20, 20), -1)
    cv2.putText(frame, status, (8, frame.shape[0] - 7),
                cv2.FONT_HERSHEY_SIMPLEX, 0.45, (180, 180, 180), 1,
cv2.LINE_AA)

    with frame_lock:
        global current_display_frame
        current_display_frame = frame

# ============================
# SPEAK
# ============================
def speak(text):
    print(f">>> {text}")
    safe_text = repr(text)
    try:
        subprocess.Popen([
            'python', '-c',
```

```
            f'import pyttsx3; e=pyttsx3.init(); e.setProperty("rate",145);
e.say({safe_text}); e.runAndWait()'
        ]).wait()
    except Exception as e:
        print("Speech Error:", e)

def get_greeting():
    hour = datetime.datetime.now().hour
    if 5 <= hour < 12:   return "Good morning! Glasses are active"
    elif 12 <= hour < 17: return "Good afternoon! Glasses are active"
    elif 17 <= hour < 21: return "Good evening! Glasses are active"
    else:                 return "Good night! Be careful in low light"

def check_low_light(image):
    gray = np.array(image.convert('L'))
    brightness = np.mean(gray)
    print(f"Brightness: {brightness:.1f}")
    return brightness < 50

def get_frame():
    try:
        response = requests.get(f"{ESP32_IP}/capture", timeout=10)
        if response.status_code == 200:
            return response.content
    except Exception as e:
        print("Camera Error:", e)
    return None

def get_ultrasonic_distance():
    try:
        response = requests.get(f"{ESP32_IP}/distance", timeout=3)
        if response.status_code == 200:
            return float(response.text.strip())
    except:
        pass
    return None

def estimate_distance_vision(label, box_height_pixels):
    if label not in KNOWN_HEIGHTS_CM or box_height_pixels < 5:
        return None
    return round((KNOWN_HEIGHTS_CM[label] * FOCAL_LENGTH) /
box_height_pixels)
```

```
def detect_objects(image_bytes):
    pil_image = PIL.Image.open(io.BytesIO(image_bytes))

    if check_low_light(pil_image):
        speak("Warning! Low light detected")
        return None, pil_image

    results = model(pil_image)
    detected = {}


    ultrasonic_dist = get_ultrasonic_distance()
    for result in results:
        for box in result.boxes:
            label = model.names[int(box.cls)]
            confidence = int(float(box.conf[0]) * 100)
            x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
            box_height_px = y2 - y1

            ultrasonic_dist = get_ultrasonic_distance()
            if ultrasonic_dist is not None:
                distance_cm = round(ultrasonic_dist)
                dist_source = "sensor"
            else:
                distance_cm = estimate_distance_vision(label, box_height_px)
                dist_source = "vision"

            if label not in detected or confidence > detected[label]['confidence']:
                detected[label] = {
                    'confidence': confidence,
                    'distance_cm': distance_cm,
                    'dist_source': dist_source,
                    'box': (x1, y1, x2, y2)
                }

    return detected, pil_image

def build_message(detected):
    if not detected:
        return None
    parts = []
```

```
    for label, info in detected.items():
        conf = info['confidence']
        dist = info['distance_cm']
        if dist:
            dist_m = dist / 100
            if dist_m < 0.5:       proximity = "very close"
            elif dist_m < 1.5:     proximity = f"{dist_m:.1f} metres away"
            else:                  proximity = f"{round(dist_m)} metres away"
            parts.append(f"{label}, {proximity}, {conf} percent sure")
        else:
            parts.append(f"{label}, {conf} percent sure")
    total = len(detected)
    if total == 1: return f"I see 1 object. {parts[0]}"
    else:          return f"I see {total} objects. " + ", ".join(parts)

# ============================
# STARTUP
# ============================
print("Testing ESP32-CAM connection...")
try:
    test = requests.get(f"{ESP32_IP}/capture", timeout=10)
    if test.status_code == 200:
        print("ESP32-CAM Connected Successfully")
    else:
        print("Bad response — check IP"); exit()
except Exception as e:
    print("Connection Failed:", e); exit()

# Start display thread BEFORE main loop
t = threading.Thread(target=display_thread, daemon=True)
t.start()
print("Camera window opened. Starting detection...\n")

speak(get_greeting())
time.sleep(1)
speak("Press R anytime to repeat last detection")
time.sleep(1)

# ============================
# MAIN LOOP
# ============================
while not stop_display:
```

```
    try:
        if keyboard.is_pressed('r'):
            if last_detected:
                speak(f"Repeating. {last_detected}")
            time.sleep(1)
            continue

        frame = get_frame()

        if frame:
            detected, pil_image = detect_objects(frame)
            update_display(pil_image, detected or {})  # always update display

            if detected:
                message = build_message(detected)
                if message and message != last_detected:
                    last_detected = message
                    speak(message)
            else:
                print("No objects detected")
        else:
            print("No frame received")

        time.sleep(3)

    except KeyboardInterrupt:
        print("\nStopping...")
        stop_display = True
        break
    except Exception as e:
        print("Runtime Error:", e)
        time.sleep(2)

cv2.destroyAllWindows()
```

Ultrasonic Sensor code

```
#define TRIG_PIN 9
#define ECHO_PIN 10
#define BUZZ_PIN 8

float getDistanceCm() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long duration = pulseIn(ECHO_PIN, HIGH, 30000);
  if (duration == 0) return -1;
  return (duration * 0.0343) / 2.0;
}

void setup() {
  Serial.begin(115200);
  delay(1000);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZ_PIN, OUTPUT);
  Serial.println("Sensor test starting...");
}

void loop() {
  float dist = getDistanceCm();
  Serial.print("Distance: ");
  Serial.print(dist);
  Serial.println(" cm");

  if (dist > 0 && dist < 40) {
    digitalWrite(BUZZ_PIN, HIGH);
    delay(100);
    digitalWrite(BUZZ_PIN, LOW);
  }

  delay(500);
}
```

Explanation of Object Detection Code

The Object Detection code is developed using Python and the YOLOv8 (You Only Look Once) deep learning model to help visually impaired users identify nearby objects in real time. The system connects to an ESP32-CAM module, captures live images, and processes them using the YOLO model to detect objects such as people, bottles, chairs, laptops, and vehicles.

The program estimates the distance of detected objects using either:

- an ultrasonic sensor connected to the ESP32, or
- computer vision calculations based on object size.

The detected objects are displayed on the screen with bounding boxes, confidence percentage, and estimated distance. A text-to-speech system announces the detected objects through audio, allowing blind users to understand their surroundings without viewing the screen.

Additional features included in the code are:

- Low-light detection and warning system
- Live camera display window
- Voice greeting system
- Repeat last detected object using keyboard input
- Real-time object tracking and distance estimation

Overall, this code acts as the main intelligent vision system of the AI-powered smart blind glasses.

Explanation of Ultrasonic Sensor Code

The Ultrasonic Sensor code is written using Arduino programming language to measure the distance between the user and nearby obstacles. The HC-SR04 ultrasonic sensor sends ultrasonic sound waves through the trigger pin and receives the reflected waves through the echo pin.

The program calculates the distance based on the time taken for the sound wave to return. If an obstacle is detected within 40 cm, the buzzer automatically turns ON to warn the user about nearby objects and prevent collisions.

Main functions of the code include:

- Generating ultrasonic pulses
- Measuring echo response time
- Calculating distance in centimeters
- Activating buzzer alerts for close obstacles
- Continuously monitoring surroundings in real time

This code provides an additional safety mechanism for obstacle detection and helps improve navigation assistance for visually impaired users.

Chapter 5

Results and Analysis

5.1 Results

The ESP32-CAM module was successfully connected to the Python application through Wi-Fi communication. The YOLOv8 object detection model identified common objects such as persons, bottles, chairs, laptops, and mobile phones with good accuracy in real-time conditions.

The ultrasonic sensor successfully measured nearby obstacle distances and activated the buzzer alert when objects were detected within the specified safety range. The text-to-speech system also generated clear voice feedback for detected objects and warning messages.

Experimental Results

Test Case	Observed Result	Status
ESP32-CAM connection	Camera stream received successfully	Successful
YOLOv8 object detection	Objects detected with bounding boxes	Successful
Voice feedback	Spoken output generated correctly	Successful
Ultrasonic sensor test	Accurate distance readings obtained	Successful
Buzzer alert	Buzzer activated for nearby obstacles	Successful
Low-light detection	Warning message generated	Successful
Live display system	Real-time display updated smoothly	Successful

5.2 Analysis

The developed system demonstrated the feasibility of combining AI-based computer vision with ultrasonic obstacle sensing in a wearable assistive device.

The YOLOv8 model provided fast and accurate object detection performance under normal lighting conditions. However, low-light environments slightly reduced detection accuracy, which was handled using a low-light warning feature.

The ultrasonic sensor provided reliable short-range distance measurement and improved obstacle awareness compared to vision-only estimation. Combining both ultrasonic sensing and AI-based detection improved the overall reliability of the system.

Advantages Observed During Testing

- Fast real-time object detection.
- Accurate obstacle distance alerts.
- Clear voice guidance for users.
- Smooth live camera visualization.
- Expandable modular architecture.

Limitations Observed

- Detection accuracy decreases in poor lighting.
- ESP32-CAM has limited processing capability.
- Wi-Fi streaming delay may occur occasionally.
- Ultrasonic sensors cannot identify object type.

5.3 User Testing and Feedback Plan

The prototype can be mounted on a spectacle frame and tested in controlled indoor and outdoor environments. Users can evaluate the effectiveness of audio alerts, obstacle detection speed, and overall comfort of the wearable system.

Suggested Testing Conditions

- Walking near walls and furniture.
- Detecting chairs, tables, and moving persons.
- Testing voice clarity in noisy environments.
- Measuring user comfort during long usage.

Feedback Parameters

Parameter	**Expected Evaluation**
Obstacle detection	User receives warning before collision
Voice clarity	Spoken feedback should be understandable
Buzzer alert	Alert intensity should be noticeable
Comfort	Glasses should remain lightweight
Safety	System should work reliably indoors

Chapter 6

Conclusion and Future Work

6.1 Conclusion

The AI Powered Blind Assistance Glasses project successfully demonstrates a low-cost wearable assistive system for visually impaired users. The system combines AI-based object detection, ultrasonic obstacle sensing, and audio feedback to provide real-time environmental awareness.

The ESP32-CAM module, YOLOv8 object detection model, and ultrasonic sensor worked together effectively to identify obstacles and guide users through voice and buzzer alerts. The project proves that affordable embedded systems and AI technologies can be integrated to develop practical accessibility solutions.

The modular architecture also allows future expansion for advanced computer vision and navigation features.

6.2 Future Work

Several improvements can be added in future versions of the project:

- Add face recognition for identifying known people.
- Integrate OCR technology for reading printed text aloud.
- Add GPS-based navigation assistance.
- Use multiple ultrasonic sensors for better directional detection.
- Improve battery backup and wearable design.
- Add wireless earphones for private voice feedback.
- Integrate cloud-based AI processing for advanced recognition tasks.

6.3 Limitations

- ESP32-CAM alone cannot perform heavy AI processing efficiently.
- Low-light conditions reduce camera detection accuracy.
- Ultrasonic sensors may detect side reflections incorrectly.
- Continuous Wi-Fi streaming may increase power consumption.
- Breadboard connections are suitable only for prototype testing.

REFERENCES

1. Ultralytics YOLOv8 Documentation.
2. OpenCV Python Documentation.
3. Espressif Systems ESP32-CAM Documentation.
4. Arduino IDE and ESP32 Board Package Documentation.
5. HC-SR04 Ultrasonic Sensor Technical Datasheet.
6. Pyttsx3 Text-to-Speech Library Documentation.
7. Python Requests Library Documentation.
8. NumPy Documentation for Image Processing.

ANNEXURES

Annexure A – Proposed Voice Messages

File Name	Message
0001.mp3	Obstacle ahead
0002.mp3	Person detected
0003.mp3	Bottle detected

0004.mp3	Warning! Low light detected
0005.mp3	Please move carefully

Annexure B – Team Roles

Role	**Responsibility**
Hardware Integration	ESP32-CAM, ultrasonic sensor, buzzer wiring
AI Development	YOLOv8 object detection and Python programming
Voice Assistant	Text-to-speech feedback system
Documentation	Report preparation and presentation

Annexure C – Notes for Final Assembly

- Keep heavy electronic components away from the front side of the glasses.
- Use compact soldered connections instead of breadboard wiring.
- Ensure all modules share a common ground connection.
- Mount the ultrasonic sensor facing forward for accurate detection.
- Use rechargeable battery modules for portable operation.